KDS Phase 4 Fix Verification Report

D01 (auto-start new order when fully sent but not bumped) · D08 (recall reference-reuse hardening) · D02 (periodic cross-terminal reconciliation)

Components under test: kds-webserver-main (nat_coms.go, datab.go, recall_order.go, reconcile.go), kds-poller-main (engine.go, database.go)

Test date: 2026-10-04

Product decision (D01): a table whose order is fully sent (every item COMPLETE and physically delivered) but never explicitly bumped must have its next ticket automatically start a new order, rather than requiring bumping as a hard prerequisite. Confirmed with the user before implementation.

1. Summary

55 of 55 unit tests passed (52 in kds-webserver-main, 3 in kds-poller-main - includes all tests from Phases 0-3 plus 19 new ones for this phase). Full `go build` / `go vet` / `go test ./...` clean across both modules.

All three fixes were additionally verified live against real running terminal processes (real NATS, real Go binaries, real raw-ticket ingestion, real HTTP peer-to-peer traffic) - including, for D02, waiting for its actual 2-minute background timer to fire on its own rather than only triggering it manually.

2. D01 — auto-start a new order when fully sent but not bumped

Implemented symmetrically in both ingestion paths: kds-poller-main's raw-ticket engine (`ProcessMessage`) and kds-webserver-main's Sally/POS-API handler (`ProcessSallyOrder`). A new `orderIsFullySentButOpen` helper (written independently in each, matching this codebase's existing convention of not sharing DB-query logic across the two binaries) checks whether every non-void/non-transferred/non-course item on an order is both `Status ==`

`"COMPLETE"` and `SentToTable == true`, with the order itself still `Status == "OPEN"` (nothing but an explicit Close Order press - `completeOrder` - ever changes that). When the next ticket for that same reference carries new items, the existing-order lookup is treated as a miss, routing it through the normal new-order path instead of silently appending to an already-finished order.

2.1 Unit tests

> `TestOrderIsFullySentButOpen_AllItemsCompleteAndSent` D01
>
> Actual: **PASS**
>
> `TestOrderIsFullySentButOpen_OneItemStillOpen` D01
>
> Actual: **PASS**
>
> `TestOrderIsFullySentButOpen_CompleteButNotYetSentToTable` D01
>
> Actual: **PASS**
>
> `TestOrderIsFullySentButOpen_OnlyVoidItemsIsNotFullySent` D01
>
> Actual: **PASS**
>
> `TestOrderIsFullySentButOpen_NoItemsAtAll` D01
>
> Actual: **PASS**
>
> `TestProcessSallyOrder_FullySentOrderStartsFreshOnNextTicketWithItems` D01
>
> Actual: **PASS**
>
> `TestProcessSallyOrder_StillOpenOrderAppendsNormally` D01 regression guard
>
> Actual: **PASS**
>
> `TestProcessMessage_FullySentOrderStartsFreshOnNextTicketWithItems` D01 (raw-ticket path)
>
> Actual: **PASS**
>
> `TestProcessMessage_StillOpenOrderAppendsNormally` D01 regression guard (raw-ticket path)
>
> Actual: **PASS**

2.2 Live verification (real raw-ticket ingestion)

> **Step 1.** Ticket round 1 (reference p4d01, table 20, "1 x burger") published to a live terminal's own poller - creates order 1.

> **Step 2.** The burger is marked COMPLETE and SentToTable directly (simulating the GUI flow), leaving the order itself OPEN - nobody presses Close Order.

> **Step 3.** Ticket round 2, same reference, "1 x pizza" published to the same terminal's poller.

Order	Status	Items
1 (original)	OPEN	**burger - COMPLETE (left untouched)**
2 (new)	OPEN	**pizza - OPEN (correctly landed on a brand new order, not appended to order 1)**

3. D08 — recall reference-reuse hardening

`getDatabaseOrderFromReferenceAnyStatus` (used only by a peer's Recall Orders fan-out, `applyRecallOrderScreenRemote`) drops the status=OPEN filter other reference lookups use - the whole point is finding a COMPLETE order to reopen. Without that filter, a reference reused later the same day could match more than one row, with only "most recently created" as a tiebreak. The recall message now carries the order's own `Table` (populated at the point the recall is first sent), and the lookup prefers whichever candidate's table matches - falling back to the original recency-only behavior when table is blank or matches nothing, so an older peer or a genuine single-match case is unaffected.

3.1 Unit tests

> **`TestGetDatabaseOrderFromReferenceAnyStatus_PrefersTableMatchOverRecency`** D08

> **Actual: PASS**

> **`TestGetDatabaseOrderFromReferenceAnyStatus_FallsBackToRecencyWhenNoTableMatch`** D08

> **Actual: PASS**

> **`TestGetDatabaseOrderFromReferenceAnyStatus_BlankTableFallsBackToRecency`** D08 regression guard

> **Actual: PASS**

> **`TestGetDatabaseOrderFromReferenceAnyStatus_TableMatchIsCaseInsensitive`** D08

> **Actual: PASS**

> **`TestGetDatabaseOrderFromReferenceAnyStatus_NoMatchingReferenceReturnsZeroValue`** D08

> **Actual: PASS**

> **`TestGetDatabaseOrderFromReferenceAnyStatus_SingleCandidateIgnoresTableMismatch`** D08 regression guard

> **Actual: PASS**

D08 is a narrow, fully-deterministic lookup change with no cross-process timing or network behavior involved - covered completely by its unit tests; no additional live reproduction was needed beyond what Phases 0-3's live mesh testing already exercises for the surrounding recall/close machinery.

4. D02 — periodic cross-terminal reconciliation

Every cross-terminal status change in this codebase (item close/open/production/send-to-table, order close, new-order/new-item sync) travels one-way and fire-and-forget over NATS, HTTP peers, and/or UDP broadcast, with no delivery guarantee - if every delivery attempt for one specific change happens to fail at once, the affected peer's copy is stuck wherever it was forever, with no later event that will ever revisit it. A new periodic pass (`asyncReconcileWithPeers`, same 2-minute-ticker idiom as the existing `asyncAnnouncePassTerminal`) now pulls each peer's current open-order snapshot (a new `/reconcilesnapshot` endpoint) and catches this terminal's own copy up on anything the peer already shows as more done - deliberately one-directional: it only ever fills in a COMPLETE or SentToTable this terminal is missing, never reverts a correct local state, using the exact same apply-side functions (`applyCloseRemoteItem`, `applySendToTableRemote`) the real-time push paths already use.

4.1 Unit tests

> **`TestReconcileSnapshotHandler_IncludesOccurrence`** D02

> **Actual: PASS**

> **`TestReconcileWithPeer_CatchesUpMissedCompletion`** D02

> **Actual: PASS**

> **`TestReconcileWithPeer_CatchesUpSentToTable`** D02

> **Actual: PASS**

> **`TestReconcileWithPeer_NeverRevertsAlreadyCompleteToOpen`** D02

> **Actual: PASS**

> **`TestReconcileWithPeer_SkipsOrderNotPresentLocally`** D02

> **Actual: PASS**

> **`TestReconcileWithPeer_UnreachablePeerIsNoop`** D02

> **Actual: PASS**

4.2 Live verification (real 2-terminal processes, real background timer)

> **Step 1.** The identical raw ticket (reference p4d02, "1 x steak") published independently to two separate live terminals' own pollers - each creates its own local copy of the order, matching the real "every terminal runs its own poller against the same feed" production architecture.

> **Step 2.** Terminal A's steak is marked COMPLETE directly in its own database - deliberately bypassing the real close endpoint/broadcast entirely, to simulate every delivery path failing for this one change.

> **Step 3.** Confirmed terminal B's copy was still OPEN at this point - the dropped-message scenario D02 targets, reproduced live.

Step 4. No manual trigger used - waited for terminal B's own real, already-running background reconciliation ticker to fire on its own schedule.

Terminal	Item status before	Item status after B's own ticker fired
A (origin)	COMPLETE	COMPLETE (unaffected)
B (never received the completion)	OPEN	**COMPLETE - self-healed automatically**

B's order status itself remained OPEN throughout (reconciliation only catches up item-level status, exactly as designed - it never force-closes an order on anyone's behalf), and none of B's or A's other unrelated orders were touched. The fix works end-to-end with zero manual intervention, confirmed against the fix's own real, unmodified background timer rather than a direct function call.

5. Conclusion

All three Phase 4 fixes - D01, D08, and D02 - pass their full unit test suite and hold under live verification against real running terminal processes. D02 in particular was confirmed to self-heal automatically via its own real background timer, not just via a manually-triggered code path. No regressions found. No unresolved issues outstanding from this phase.